VSP TECHVERSE
An Institute of AI & Robotics

Python Fundamentals for AI & Robotics



Functions · Loops · Conditions · Error Handling · Strings · VirtualEnv — with AI & Robotics examples

VSP TECHVERSE
An Institute of AI & Robotics

def

Python Functions

Reusable blocks of code that run only when called, such as starting a motor or reading a sensor.

TOPICS IN THIS SECTION

1. Python Functions
2. Python Arguments
3. Python Decorators
4. Python Lambda
5. Python Recursion
6. Python Generators

Python Functions

example.py

```
def start_motor():
  print("Motor started")

start_motor()
start_motor()
```

OUTPUT

```
Motor started
Motor started
```

KEY POINTS

- A function is a block of code that runs only when it is called
- Define it with the def keyword
- Call it by writing its name followed by parentheses

KEY NOTE

A function can be called as many times as you need.

Python Arguments

example.py

```
def move_joint(joint):
  print(joint + " moved")

move_joint("base")
move_joint("elbow")
move_joint("wrist")
```

OUTPUT

```
base moved
elbow moved
wrist moved
```

KEY POINTS

- Information is passed into functions as arguments
- Arguments are listed inside the parentheses
- A function can take as many arguments as you need

KEY NOTE

Parameters are the names in the definition; arguments are the values you pass in.

Python Decorators

example.py

```
def announce(func):
  def wrapper():
    print("Command received")
    func()
  return wrapper

@announce
def move_forward():
  print("Robot moving forward")

move_forward()
```

OUTPUT

```
Command received
Robot moving forward
```

KEY POINTS

- A decorator adds behavior to a function without changing its code
- Write @decorator_name above the function to apply it
- The decorator wraps the original function in a new one

KEY NOTE

Decorators let you modify a function's behavior without editing it.

Python Lambda

example.py

```
add_offset = lambda speed: speed + 10
print(add_offset(5))

torque = lambda force, radius: force * radius
print(torque(5, 6))
```

OUTPUT

```
15
30
```

KEY POINTS

- A lambda is a small anonymous function
- It can take any number of arguments but has only one expression
- Written as: lambda arguments : expression

KEY NOTE

Use lambda when you need a simple function for a short time.

Python Recursion

example.py

```
def countdown(n):
  if n == 0:
    print("Drone takeoff!")
  else:
    print(n)
    countdown(n - 1)

countdown(3)
```

OUTPUT

```
3
2
1
Drone takeoff!
```

KEY POINTS

- Recursion means a function calls itself
- It needs a stopping condition, or it never ends
- Great for problems that break into smaller copies of themselves

KEY NOTE

Always make sure a recursive function has a condition that ends it.

Python Generators

example.py

```
def sensor_readings():
  yield 12.5
  yield 13.1
  yield 12.9

for value in sensor_readings():
  print(value)
```

OUTPUT

```
12.5
13.1
12.9
```

KEY POINTS

- A generator produces values one at a time using yield
- It pauses after each yield and resumes where it left off
- Saves memory because values are made only when needed

KEY NOTE

Generators are ideal for streams of data such as sensor readings.



Control Flow

Loops and conditions let a robot repeat actions, make decisions and survive errors.

TOPICS IN THIS SECTION

1. Python While Loops
2. Python For Loops
3. Python If...Else
4. Python Try...Except

Python While Loops

example.py

```
level = 1
while level < 6:
  print("Charge level", level)
  level += 1
```

OUTPUT

```
Charge level 1
Charge level 2
Charge level 3
Charge level 4
Charge level 5
```

KEY POINTS

- A while loop runs as long as its condition is true
- The loop variable must be updated, or the loop never ends
- break and continue control the loop from inside

KEY NOTE

Remember to increase the counter, or the loop will continue forever.

Python For Loops

example.py

```
sensors = ["lidar", "camera", "imu"]

for x in sensors:
  print(x)

for n in range(3):
  print("Scan", n)
```

OUTPUT

```
lidar
camera
imu
Scan 0
Scan 1
Scan 2
```

KEY POINTS

- A for loop iterates over a list, tuple, string or range
- It works with any sequence, one item at a time
- range() repeats an action a set number of times

KEY NOTE

The for loop does not need an indexing variable set in advance.

Python If...Else

example.py

```
battery = 15
limit = 20

if battery > limit:
  print("Battery OK")
elif battery == limit:
  print("Battery at limit")
else:
  print("Recharge needed")
```

OUTPUT

```
Recharge needed
```

KEY POINTS

- if runs code when a condition is true
- elif tries another condition if the first was false
- else catches everything that is left

KEY NOTE

Python relies on indentation to define the code blocks.

Python Try...Except

example.py

```
try:
  print(distance)
except NameError:
  print("Sensor value is missing")
finally:
  print("Check complete")
```

OUTPUT

```
Sensor value is missing
Check complete
```

KEY POINTS

- try holds code that might raise an error
- except handles the error so the program keeps running
- finally runs no matter what happened

KEY NOTE

Without try...except, an error would stop the program.



Strings & Tools

Format messages for robot logs, and keep each project's packages in its own environment.

TOPICS IN THIS SECTION

1. Python String Formatting
2. Python VirtualEnv

Python String Formatting

example.py

```
name = "Rover-2"
battery = 85.456

txt = f"{name} battery: {battery:.1f}%"
print(txt)

print("Speed: {} m/s".format(2.5))
```

OUTPUT

```
Rover-2 battery: 85.5%
Speed: 2.5 m/s
```

KEY POINTS

- f-strings put values inside text using curly braces
- Add a format spec such as :.2f to control decimals
- format() is an older way to do the same job

KEY NOTE

An f-string starts with f and placeholders go inside {}.

Python VirtualEnv

example.py

```
# create the environment
python -m venv robot_env

# activate it (Windows)
robot_env\Scripts\activate

# install a package inside it
pip install numpy

# leave the environment
deactivate
```

OUTPUT

```
(robot_env) C:\robots> pip install numpy
Successfully installed numpy
(robot_env) C:\robots> deactivate
C:\robots>
```

KEY POINTS

- A virtual environment keeps one project's packages separate from others
- Create it with python -m venv, then activate it
- Install packages inside it, and deactivate when done

KEY NOTE

Each robot project can have its own package versions.

Thank You

Python fundamentals, with examples adapted for AI & Robotics



VSP TECHVERSE
An Institute of AI & Robotics